

```
In [1]: # Import Required Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: # Load cleaned dataset

df_final = pd.read_csv("E:\Quantities_Data_Analyst_Project\Task-
2\QVI_data.csv")
```

```
In [3]: df_final.head()
```

```
Out[3]:
```

|   | LYLTY_CARD_NBR | DATE       | STORE_NBR | TXN_ID | PROD_NBR | PROD_NAME                             | PROD_QTY | TOT_SAL |
|---|----------------|------------|-----------|--------|----------|---------------------------------------|----------|---------|
| 0 | 1000           | 17-10-2018 | 1         | 1      | 5        | Natural Chip Compny SeaSalt175g       | 2        |         |
| 1 | 1002           | 16-09-2018 | 1         | 2      | 58       | Red Rock Deli Chikn&Garlic Aioli 150g | 1        |         |
| 2 | 1003           | 07-03-2019 | 1         | 3      | 52       | Grain Waves Sour Cream&Chives 210G    | 1        |         |
| 3 | 1003           | 08-03-2019 | 1         | 4      | 106      | Natural ChipCo Hony Soy Chckn175g     | 1        |         |
| 4 | 1004           | 02-11-2018 | 1         | 5      | 96       | WW Original Stacked Chips 160g        | 1        |         |

```
In [4]: df_final.rename(columns = str.lower,inplace = True)
```

```
In [5]: df_final.head(2)
```

```
Out[5]:
```

|   | lylty_card_nbr | date       | store_nbr | txn_id | prod_nbr | prod_name                             | prod_qty | tot_sales | pack_size |
|---|----------------|------------|-----------|--------|----------|---------------------------------------|----------|-----------|-----------|
| 0 | 1000           | 17-10-2018 | 1         | 1      | 5        | Natural Chip Compny SeaSalt175g       | 2        | 6.0       | 175       |
| 1 | 1002           | 16-09-2018 | 1         | 2      | 58       | Red Rock Deli Chikn&Garlic Aioli 150g | 1        | 2.7       | 150       |

In [6]: `# Convert date column into datetime format`

```
df_final["date"] = pd.to_datetime(
    df_final["date"],
    format="%d-%m-%Y"
)
df_final['month_id'] = df_final['date'].dt.to_period('M')
```

In [7]: `df_final.head()`

Out[7]:

|   | lylty_card_nbr | date       | store_nbr | txn_id | prod_nbr | prod_name                                   | prod_qty | tot_sales | pack_size |
|---|----------------|------------|-----------|--------|----------|---------------------------------------------|----------|-----------|-----------|
| 0 | 1000           | 2018-10-17 | 1         | 1      | 5        | Natural Chip<br>Compny<br>SeaSalt175g       | 2        | 6.0       | 175       |
| 1 | 1002           | 2018-09-16 | 1         | 2      | 58       | Red Rock Deli<br>Chikn&Garlic<br>Aioli 150g | 1        | 2.7       | 150       |
| 2 | 1003           | 2019-03-07 | 1         | 3      | 52       | Grain Waves<br>Sour<br>Cream&Chives<br>210G | 1        | 3.6       | 210       |
| 3 | 1003           | 2019-03-08 | 1         | 4      | 106      | Natural<br>ChipCo Hony<br>Soy<br>Chckn175g  | 1        | 3.0       | 175       |
| 4 | 1004           | 2018-11-02 | 1         | 5      | 96       | WW Original<br>Stacked Chips<br>160g        | 1        | 1.9       | 160       |

In [8]: `"""Create Monthly Metrics  
1. Total Sales  
2. Number of Customers  
3. Average Transactions per Customer  
  
We aggregate store-month level"""`

Out[8]: `'Create Monthly Metrics\n1. Total Sales\n2. Number of Customers\n3. Average Transactions per Customer\n\nWe aggregate store-month level'`

In [9]: `df_final.columns`

Out[9]: `Index(['lylty_card_nbr', 'date', 'store_nbr', 'txn_id', 'prod_nbr',  
'prod_name', 'prod_qty', 'tot_sales', 'pack_size', 'brand', 'lifestage',  
'premium_customer', 'month_id'],  
dtype='object')`

In [10]:

```
# Aggregate monthly metrics for each store

monthly_matrix = df_final.groupby(['store_nbr', 'month_id']).agg(
    Total_sales = ('tot_sales', 'sum'),
    no_of_cust = ('lylty_card_nbr', 'nunique'),
    transaction = ('txn_id', 'nunique')).reset_index()

monthly_matrix['Avg_txn_per_cust'] =
monthly_matrix['transaction']/monthly_matrix['no_of_cust']

monthly_matrix
```

Out[10]:

|      | store_nbr | month_id | Total_sales | no_of_cust | transaction | Avg_txn_per_cust |
|------|-----------|----------|-------------|------------|-------------|------------------|
| 0    | 1         | 2018-07  | 206.9       | 49         | 52          | 1.061224         |
| 1    | 1         | 2018-08  | 176.1       | 42         | 43          | 1.023810         |
| 2    | 1         | 2018-09  | 278.8       | 59         | 62          | 1.050847         |
| 3    | 1         | 2018-10  | 188.1       | 44         | 45          | 1.022727         |
| 4    | 1         | 2018-11  | 192.6       | 46         | 47          | 1.021739         |
| ...  | ...       | ...      | ...         | ...        | ...         | ...              |
| 3164 | 272       | 2019-02  | 395.5       | 45         | 48          | 1.066667         |
| 3165 | 272       | 2019-03  | 442.3       | 50         | 53          | 1.060000         |
| 3166 | 272       | 2019-04  | 445.1       | 54         | 55          | 1.018519         |
| 3167 | 272       | 2019-05  | 314.6       | 34         | 40          | 1.176471         |
| 3168 | 272       | 2019-06  | 312.1       | 34         | 37          | 1.088235         |

3169 rows × 6 columns

In [11]:

```
monthly_matrix.describe()
```

Out[11]:

|       | store_nbr   | Total_sales | no_of_cust  | transaction | Avg_txn_per_cust |
|-------|-------------|-------------|-------------|-------------|------------------|
| count | 3169.000000 | 3169.000000 | 3169.000000 | 3169.000000 | 3169.000000      |
| mean  | 136.802461  | 610.007889  | 69.826128   | 83.031556   | 1.141307         |
| std   | 78.418604   | 389.699478  | 36.817352   | 47.918428   | 0.111341         |
| min   | 1.000000    | 1.500000    | 1.000000    | 1.000000    | 1.000000         |
| 25%   | 68.000000   | 260.000000  | 41.000000   | 43.000000   | 1.041667         |
| 50%   | 137.000000  | 674.600000  | 79.000000   | 96.000000   | 1.125000         |
| 75%   | 204.000000  | 928.200000  | 102.000000  | 127.000000  | 1.233645         |
| max   | 272.000000  | 1659.600000 | 150.000000  | 188.000000  | 1.468085         |

```
In [12]: # Define Pre-Trial Period
# trail start from Feb'2019 so control period will be before feb'19
```

```
In [13]: pre_trial = monthly_matrix[monthly_matrix['month_id'] < '2019-02']
pre_trial.head()
```

```
Out[13]:
```

|   | store_nbr | month_id | Total_sales | no_of_cust | transaction | Avg_txn_per_cust |
|---|-----------|----------|-------------|------------|-------------|------------------|
| 0 | 1         | 2018-07  | 206.9       | 49         | 52          | 1.061224         |
| 1 | 1         | 2018-08  | 176.1       | 42         | 43          | 1.023810         |
| 2 | 1         | 2018-09  | 278.8       | 59         | 62          | 1.050847         |
| 3 | 1         | 2018-10  | 188.1       | 44         | 45          | 1.022727         |
| 4 | 1         | 2018-11  | 192.6       | 46         | 47          | 1.021739         |

```
In [14]: from scipy.stats import pearsonr
import numpy as np
import pandas as pd

def calculate_correlation(metric, trial_store):

    # Data for the trial store
    trial = pre_trial[
        pre_trial["store_nbr"] == trial_store
    ][["month_id", metric]]

    correlations = {}

    # Compare with every other store
    for store in pre_trial["store_nbr"].unique():

        if store == trial_store:
            continue

        control = pre_trial[
            pre_trial["store_nbr"] == store
        ][["month_id", metric]]

        # Merge on month
        merged = pd.merge(
            trial,
            control,
```

```

        on="month_id",
        suffixes=("_trial", "_control")
    )

    # Skip stores with missing months
    if len(merged) != len(trial):
        continue

    trial_values = merged[f"{metric}_trial"]
    control_values = merged[f"{metric}_control"]

    # Pearson correlation cannot be computed if values are constant
    if trial_values.nunique() <= 1 or control_values.nunique() <= 1:
        correlations[store] = np.nan
    else:
        corr, _ = pearsonr(trial_values, control_values)
        correlations[store] = corr

    return pd.Series(correlations, name=f"{metric}_corr")

```

```

In [15]: Avg_txn_per_cust_corr = calculate_correlation('Avg_txn_per_cust',77)
Avg_txn_per_cust_corr.sort_values(ascending=False).head(10)

```

```

Out[15]: 176    0.881098
144    0.868818
86     0.850527
118    0.817083
181    0.813600
119    0.802529
213    0.763779
13     0.758140
190    0.751048
271    0.717741
Name: Avg_txn_per_cust_corr, dtype: float64

```

```

In [16]: # total sales correlations

sales_corr = calculate_correlation('Total_sales',77)
sales_corr.sort_values(ascending=False).head(10)

```

```
Out[16]: 71    0.914106
233    0.903774
119    0.867664
17     0.842668
3      0.806644
41     0.783232
50     0.763866
157    0.735893
162    0.729740
257    0.724927
Name: Total_sales_corr, dtype: float64
```

```
In [17]: # no_of_cust correlation

no_of_cust_corr = calculate_correlation('no_of_cust',77)
no_of_cust_corr.sort_values(ascending=False).head(10)
```

```
Out[17]: 233    0.990358
119    0.983267
254    0.916208
113    0.901348
84     0.858571
41     0.844219
3      0.834207
35     0.774647
88     0.765048
26     0.759452
Name: no_of_cust_corr, dtype: float64
```

```
In [18]: monthly_matrix.head()
```

```
Out[18]:
```

|   | store_nbr | month_id | Total_sales | no_of_cust | transaction | Avg_txn_per_cust |
|---|-----------|----------|-------------|------------|-------------|------------------|
| 0 | 1         | 2018-07  | 206.9       | 49         | 52          | 1.061224         |
| 1 | 1         | 2018-08  | 176.1       | 42         | 43          | 1.023810         |
| 2 | 1         | 2018-09  | 278.8       | 59         | 62          | 1.050847         |
| 3 | 1         | 2018-10  | 188.1       | 44         | 45          | 1.022727         |
| 4 | 1         | 2018-11  | 192.6       | 46         | 47          | 1.021739         |

```
In [19]: pre_trial.columns
```

```
Out[19]: Index(['store_nbr', 'month_id', 'Total_sales', 'no_of_cust', 'transaction',
        'Avg_txn_per_cust'],
        dtype='object')
```

```
In [20]: def calculate_magnitude(metric, trial_store):
```

```
    # Data for the trial store
    trial = pre_trial[
        pre_trial["store_nbr"] == trial_store
```

```
][["month_id", metric]]

magnitude = {}

# Compare with every other store
for store in pre_trial["store_nbr"].unique():

    if store == trial_store:
        continue

    control = pre_trial[
        pre_trial["store_nbr"] == store
    ][["month_id", metric]]

    merged = pd.merge(
        trial,
        control,
        on="month_id",
        suffixes=("_trial", "_control")
    )

    # Skip stores with missing months
    if len(merged) != len(trial):
        continue

    diff = abs(
        merged[f"{metric}_trial"] -
        merged[f"{metric}_control"]
    )

    # Handle constant differences
    if diff.max() == diff.min():
        magnitude_score = 1
    else:
        score = 1 - (
            (diff - diff.min()) /
            (diff.max() - diff.min())
        )
        magnitude_score = score.mean()

    magnitude[store] = magnitude_score
```

```
return pd.Series(magnitude, name=f"{metric}_magn")
```

```
In [21]: Total_sales_mag = calculate_magnitude('Total_sales',77)
Total_sales_mag.sort_values(ascending = False).head(10)
```

```
Out[21]: 18      0.733070
115     0.727536
69      0.720729
43      0.715421
118     0.712821
51      0.711352
214     0.698926
192     0.690417
247     0.684843
114     0.679845
Name: Total_sales_magn, dtype: float64
```

```
In [22]: no_of_cust_mag = calculate_magnitude('no_of_cust',77)
no_of_cust_mag.sort_values(ascending = False).head(10)
```

```
Out[22]: 260     0.757764
64      0.740260
48      0.730159
51      0.727891
171     0.714286
56      0.709360
36      0.708791
19      0.707641
265     0.705882
124     0.700000
Name: no_of_cust_magn, dtype: float64
```

```
In [23]: Avg_txn_per_cust_mag = calculate_magnitude('Avg_txn_per_cust',77)
Avg_txn_per_cust_mag.sort_values(ascending = False).head(10)
```

```
Out[23]: 260     0.786707
254     0.754644
9       0.749375
200     0.748203
46      0.746348
228     0.737893
234     0.731308
269     0.724397
25      0.706615
233     0.699250
Name: Avg_txn_per_cust_magn, dtype: float64
```

```
In [24]: monthly_matrix.columns
```

```
Out[24]: Index(['store_nbr', 'month_id', 'Total_sales', 'no_of_cust', 'transaction',
              'Avg_txn_per_cust'],
              dtype='object')
```



In [25]:

```
def get_similarity(trial_store):

    sales_corr = calculate_correlation("Total_sales", trial_store)
    customer_corr = calculate_correlation("no_of_cust", trial_store)
    txn_corr = calculate_correlation("Avg_txn_per_cust", trial_store)

    sales_magn = calculate_magnitude("Total_sales", trial_store)
    customer_magn = calculate_magnitude("no_of_cust", trial_store)
    txn_magn = calculate_magnitude("Avg_txn_per_cust", trial_store)

    similarity = pd.concat(
        [
            sales_corr,
            customer_corr,
            txn_corr,
            sales_magn,
            customer_magn,
            txn_magn,
        ],
        axis=1
    )

    similarity.columns = [
        "sales_corr",
        "customer_corr",
        "txn_corr",
        "sales_magn",
        "customer_magn",
        "txn_magn"
    ]

    # Remove stores with undefined transaction correlation
    similarity = similarity.dropna(subset=["txn_corr"])

    # Convert index into a column
    similarity = similarity.reset_index()
    similarity.rename(columns={"index": "store_nbr"}, inplace=True)

    # Calculate average score for each metric
    similarity["sales_score"] = (
        similarity["sales_corr"] +
```

```

        similarity["sales_magn"]
    ) / 2

    similarity["customer_score"] = (
        similarity["customer_corr"] +
        similarity["customer_magn"]
    ) / 2

    similarity["txn_score"] = (
        similarity["txn_corr"] +
        similarity["txn_magn"]
    ) / 2

    # Overall similarity score
    similarity["Final_Score"] = similarity[
        ["sales_score", "customer_score", "txn_score"]
    ].mean(axis=1)

    return similarity.sort_values(
        "Final_Score",
        ascending=False
    )

```

```

In [26]: similarity_77 = get_similarity(77)
        similarity_86 = get_similarity(86)
        similarity_88 = get_similarity(88)

```

```

In [27]: similarity_88.head(10)

```

```

Out[27]:

```

|     | store_nbr | sales_corr | customer_corr | txn_corr  | sales_magn | customer_magn | txn_magn | sales |
|-----|-----------|------------|---------------|-----------|------------|---------------|----------|-------|
| 158 | 178       | 0.731857   | 0.939466      | 0.333328  | 0.569749   | 0.476190      | 0.635275 | 0.6   |
| 179 | 201       | 0.492735   | 0.511302      | 0.707710  | 0.600369   | 0.507937      | 0.625233 | 0.9   |
| 100 | 113       | 0.495763   | 0.862632      | 0.555803  | 0.414447   | 0.428571      | 0.565559 | 0.4   |
| 224 | 253       | 0.811838   | 0.500962      | 0.259258  | 0.521990   | 0.540816      | 0.538176 | 0.6   |
| 167 | 187       | 0.599076   | 0.671264      | 0.343941  | 0.634428   | 0.517857      | 0.392512 | 0.6   |
| 119 | 134       | 0.864293   | 0.508880      | -0.015341 | 0.685875   | 0.571429      | 0.523414 | 0.7   |
| 93  | 106       | 0.644724   | 0.200482      | 0.803873  | 0.479495   | 0.402116      | 0.568051 | 0.9   |
| 129 | 145       | 0.450400   | 0.480189      | 0.671867  | 0.339634   | 0.583333      | 0.553307 | 0.3   |
| 28  | 32        | 0.234299   | 0.532606      | 0.665810  | 0.583253   | 0.444444      | 0.540587 | 0.4   |
| 37  | 41        | 0.290319   | 0.721650      | 0.306447  | 0.562729   | 0.650794      | 0.391049 | 0.4   |

In [28]:

```
import seaborn as sns
def visual_similarity(trial_store):

    top5 = (
        get_similarity(trial_store)
        .sort_values("Final_Score", ascending=False)
        .head()
    )

    plt.figure(figsize=(8,5))

    sns.barplot(
        data=top5.reset_index(),
        x="store_nbr",
        y="Final_Score"
    )

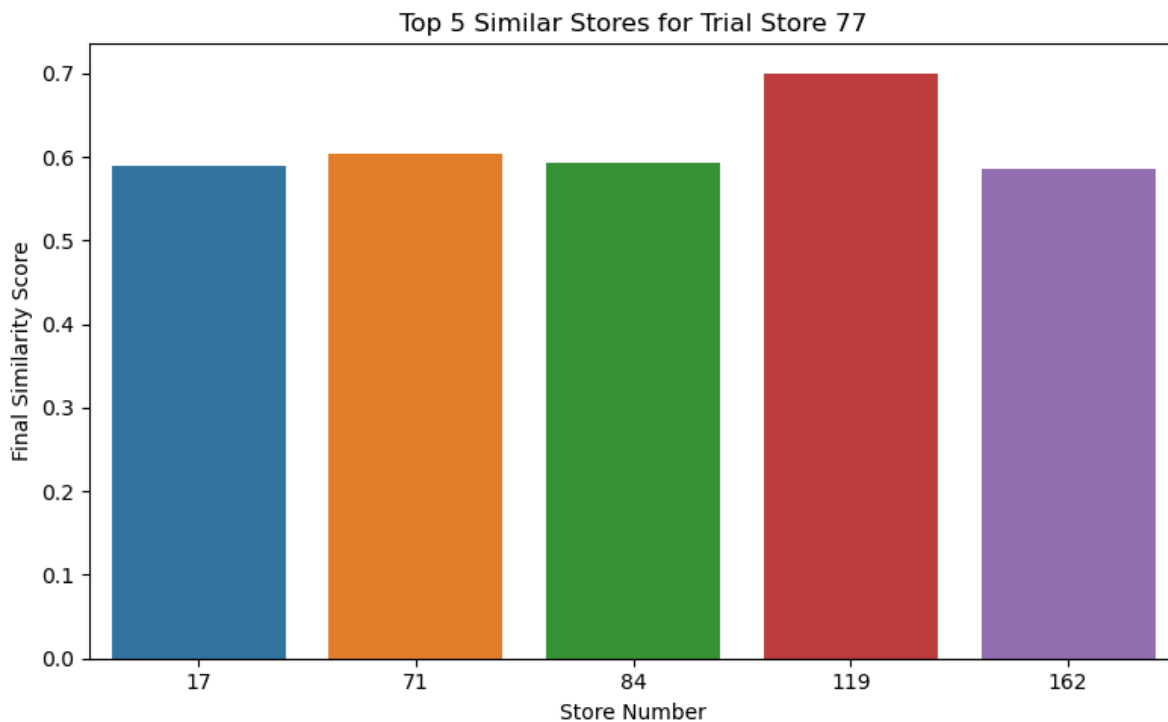
    plt.title(f"Top 5 Similar Stores for Trial Store {trial_store}")
    plt.xlabel("Store Number")
    plt.ylabel("Final Similarity Score")

    plt.tight_layout()
    plt.show()

    return top5
```

In [29]:

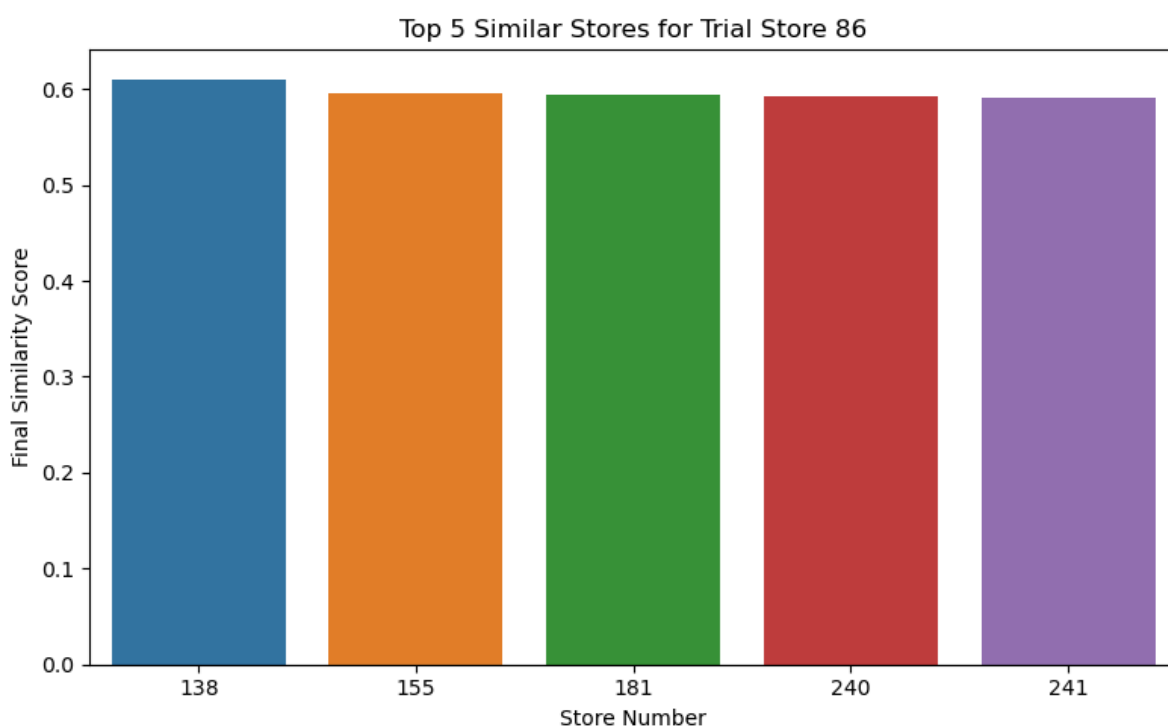
```
top5_77 = visual_similarity(77)
top5_77
```



Out[29]:

|     | store_nbr | sales_corr | customer_corr | txn_corr | sales_magn | customer_magn | txn_magn | sales_ |
|-----|-----------|------------|---------------|----------|------------|---------------|----------|--------|
| 105 | 119       | 0.867664   | 0.983267      | 0.802529 | 0.404428   | 0.476190      | 0.666546 | 0.6    |
| 63  | 71        | 0.914106   | 0.754817      | 0.256314 | 0.664887   | 0.571429      | 0.465046 | 0.7    |
| 74  | 84        | 0.684348   | 0.858571      | 0.396508 | 0.629596   | 0.571429      | 0.418741 | 0.6    |
| 14  | 17        | 0.842668   | 0.747308      | 0.423245 | 0.409456   | 0.571429      | 0.544497 | 0.6    |
| 143 | 162       | 0.729740   | 0.688577      | 0.619355 | 0.490842   | 0.563910      | 0.417860 | 0.6    |

```
In [30]: top5_86 = visual_similarity(86)
top5_86
```



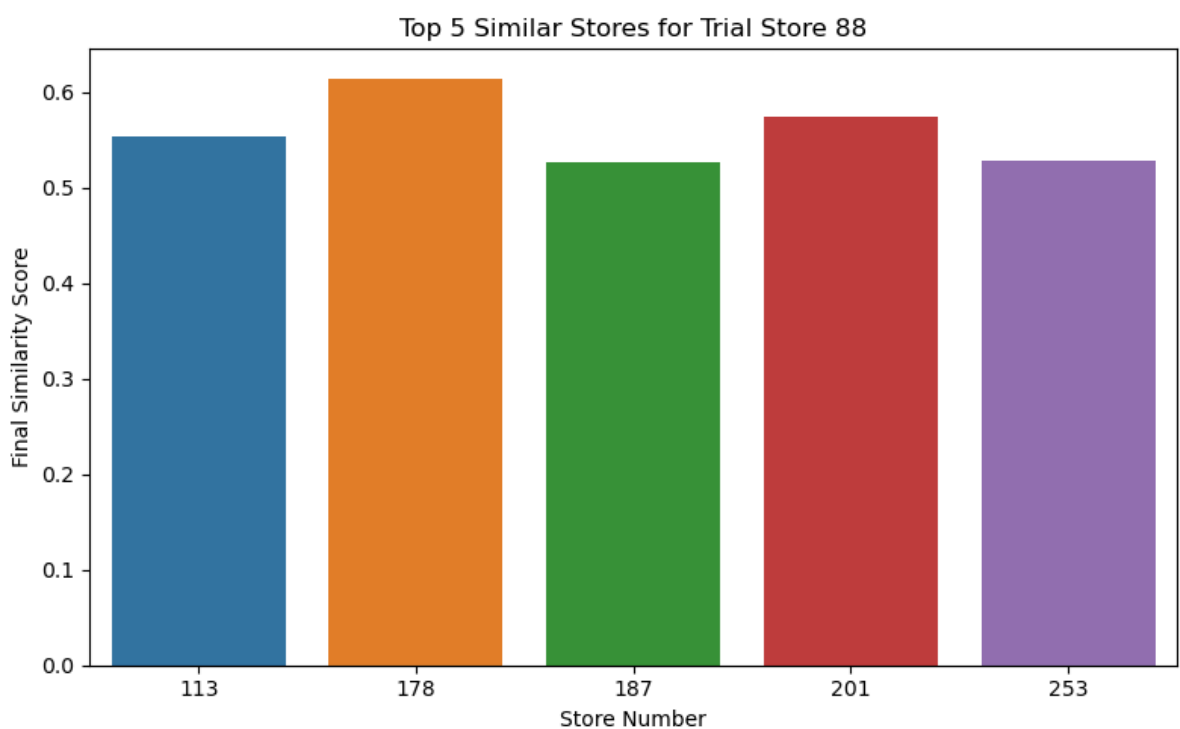
Out[30]:

|            | store_nbr | sales_corr | customer_corr | txn_corr  | sales_magn | customer_magn | txn_magn | sales |
|------------|-----------|------------|---------------|-----------|------------|---------------|----------|-------|
| <b>123</b> | 138       | 0.759864   | 0.749701      | 0.543144  | 0.426729   | 0.571429      | 0.611028 | 0.9   |
| <b>138</b> | 155       | 0.877882   | 0.942876      | -0.070774 | 0.738330   | 0.523810      | 0.560820 | 0.8   |
| <b>161</b> | 181       | 0.607657   | 0.549331      | 0.652390  | 0.571478   | 0.673469      | 0.505535 | 0.9   |
| <b>213</b> | 240       | 0.825066   | 0.576403      | 0.609682  | 0.409573   | 0.610390      | 0.523326 | 0.6   |
| <b>214</b> | 241       | 0.282051   | 0.538847      | 0.515786  | 0.758264   | 0.714286      | 0.735882 | 0.9   |

In [ ]:

In [31]:

```
top5_88 = visual_similarity(88)
top5_88
```



Out[31]:

|            | store_nbr | sales_corr | customer_corr | txn_corr | sales_magn | customer_magn | txn_magn | sales |
|------------|-----------|------------|---------------|----------|------------|---------------|----------|-------|
| <b>158</b> | 178       | 0.731857   | 0.939466      | 0.333328 | 0.569749   | 0.476190      | 0.635275 | 0.6   |
| <b>179</b> | 201       | 0.492735   | 0.511302      | 0.707710 | 0.600369   | 0.507937      | 0.625233 | 0.5   |
| <b>100</b> | 113       | 0.495763   | 0.862632      | 0.555803 | 0.414447   | 0.428571      | 0.565559 | 0.4   |
| <b>224</b> | 253       | 0.811838   | 0.500962      | 0.259258 | 0.521990   | 0.540816      | 0.538176 | 0.6   |
| <b>167</b> | 187       | 0.599076   | 0.671264      | 0.343941 | 0.634428   | 0.517857      | 0.392512 | 0.6   |

In [32]:

```
def control_store_find(trial_store):
    control_score = get_similarity(trial_store).iloc[0]["store_nbr"]
    #print(f"Trial Store {trial_store} -> Control Store:,
    {control_score}")
    return control_score
```

```
In [33]: control_store_find(77)
```

```
Out[33]: 119.0
```

```
In [34]: def compare_trial_control(trial_store):
          control_store = control_store_find(trial_store)
          trial = monthly_matrix[
              monthly_matrix["store_nbr"] == trial_store
          ]

          control = monthly_matrix[
              monthly_matrix["store_nbr"] == control_store
          ]

          comparison = trial.merge(
              control,
              on="month_id",
              suffixes=("_trial", "_control")
          )

          return comparison
```

```
In [35]: comparison_77 = compare_trial_control(77)
          comparison_77.head()
```

```
Out[35]:
```

|   | store_nbr_trial | month_id | Total_sales_trial | no_of_cust_trial | transaction_trial | Avg_txn_per_cust_trial |
|---|-----------------|----------|-------------------|------------------|-------------------|------------------------|
| 0 | 77              | 2018-07  | 296.8             | 51               | 55                | 1.0784                 |
| 1 | 77              | 2018-08  | 255.5             | 47               | 48                | 1.0212                 |
| 2 | 77              | 2018-09  | 225.2             | 42               | 44                | 1.0476                 |
| 3 | 77              | 2018-10  | 204.5             | 37               | 38                | 1.0270                 |
| 4 | 77              | 2018-11  | 245.3             | 41               | 44                | 1.0731                 |

```
In [36]: comparison_86 = compare_trial_control(86)
          comparison_86.head()
```

Out[36]:

|   | store_nbr_trial | month_id | Total_sales_trial | no_of_cust_trial | transaction_trial | Avg_txn_per_cust_trial |
|---|-----------------|----------|-------------------|------------------|-------------------|------------------------|
| 0 | 86              | 2018-07  | 892.20            | 99               | 126               | 1.2727                 |
| 1 | 86              | 2018-08  | 764.05            | 94               | 110               | 1.1702                 |
| 2 | 86              | 2018-09  | 914.60            | 103              | 128               | 1.2427                 |
| 3 | 86              | 2018-10  | 948.40            | 109              | 138               | 1.2660                 |
| 4 | 86              | 2018-11  | 918.00            | 100              | 125               | 1.2500                 |

In [37]:

```
def plot_trial_control_metric(trial_store, metric, title, ylabel):
```

```
    comparison = compare_trial_control(trial_store)
    control_store = control_store_find(trial_store)
```

```
    plt.figure(figsize=(12,6))
```

```
    plt.plot(
        comparison["month_id"].astype(str),
        comparison[f"{metric}_trial"],
        marker="o",
        linewidth=2,
        label=f"Trial Store {trial_store}"
    )
```

```
    plt.plot(
        comparison["month_id"].astype(str),
        comparison[f"{metric}_control"],
        marker="o",
        linewidth=2,
        label=f"Control Store {control_store}"
    )
```

```
    plt.axvline(
        x=6,
        color="black",
        linestyle="--",
        label="Trial Begins"
    )
```

```
    plt.title(
        f"{title}\nTrial Store {trial_store} vs Control Store {control_store}"
    )
```

```
)

plt.xlabel("Month")
plt.ylabel(ylabel)

plt.xticks(rotation=45)

plt.legend()

plt.tight_layout()
plt.tight_layout()
plt.savefig(
    f"Store_{trial_store}_{metric}.png",
    dpi=300,
    bbox_inches="tight")

plt.show()
```

In [38]: `for store in [77, 86, 88]:`

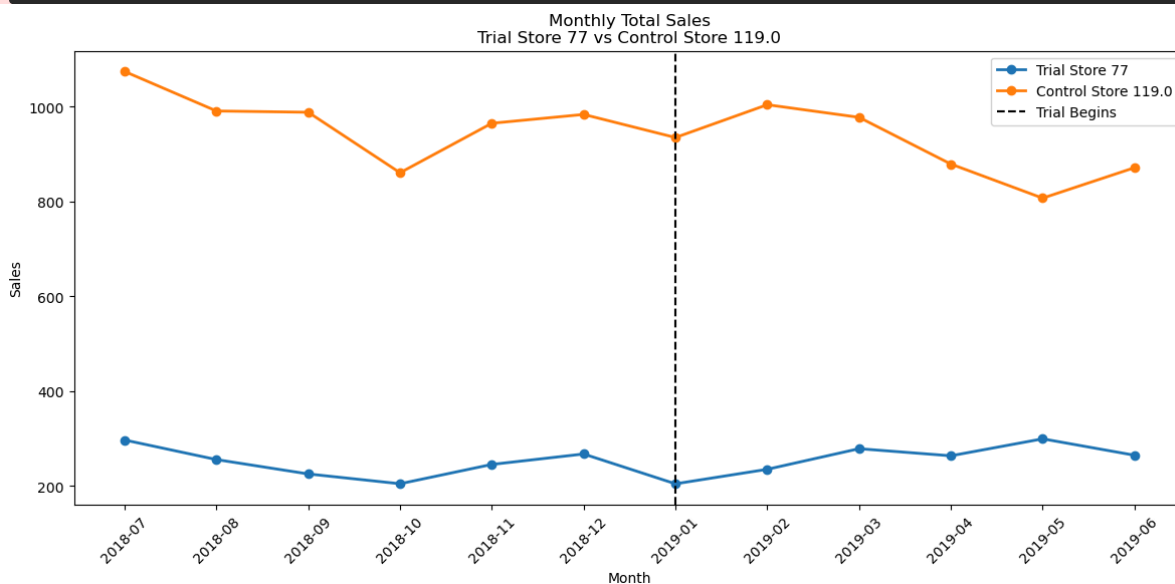
```
    plot_trial_control_metric(
        store,
        "Total_sales",
        "Monthly Total Sales",
        "Sales"
    )

    plot_trial_control_metric(
        store,
        "no_of_cust",
        "Monthly Number of Customers",
        "Customers"
    )

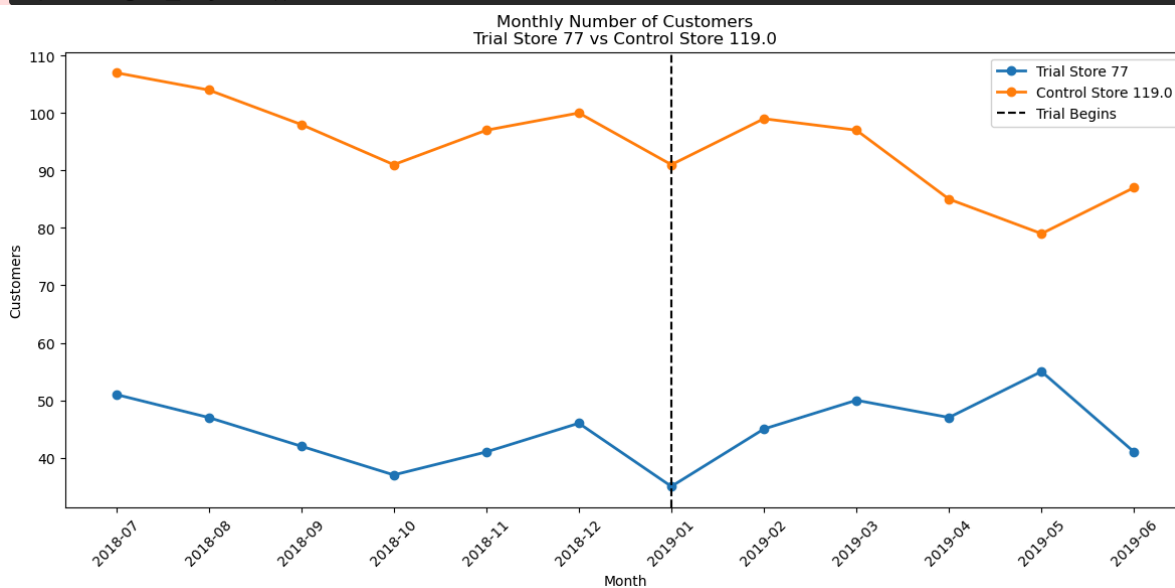
    plot_trial_control_metric(
        store,
        "Avg_txn_per_cust",
        "Monthly Average Transactions per Customer",
        "Transactions per Customer"
    )
```



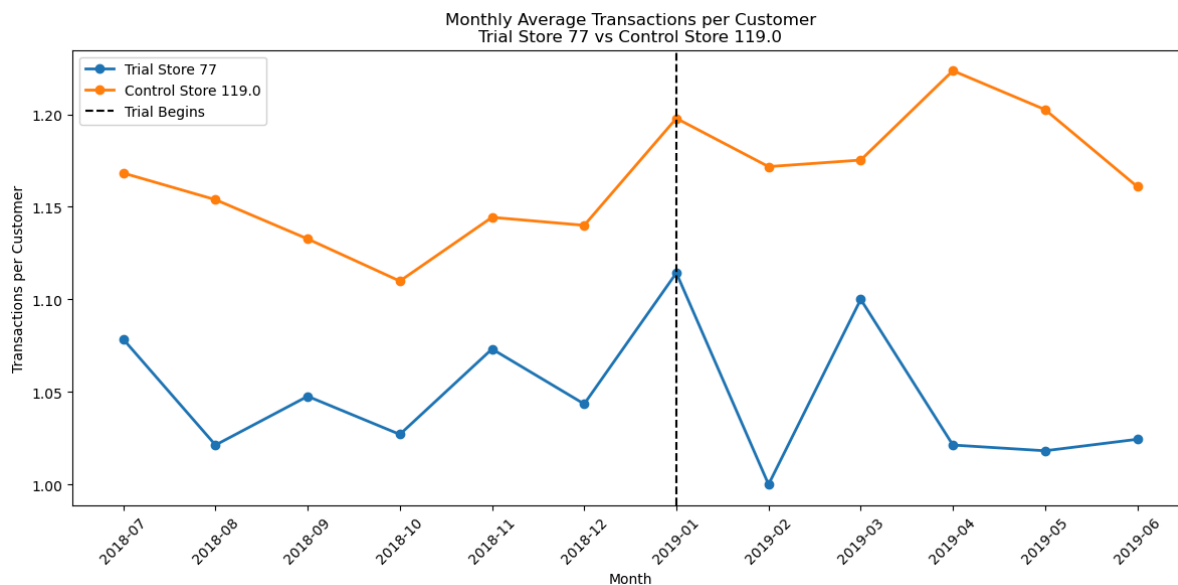
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:  
The figure layout has changed to tight  
plt.tight_layout()
```



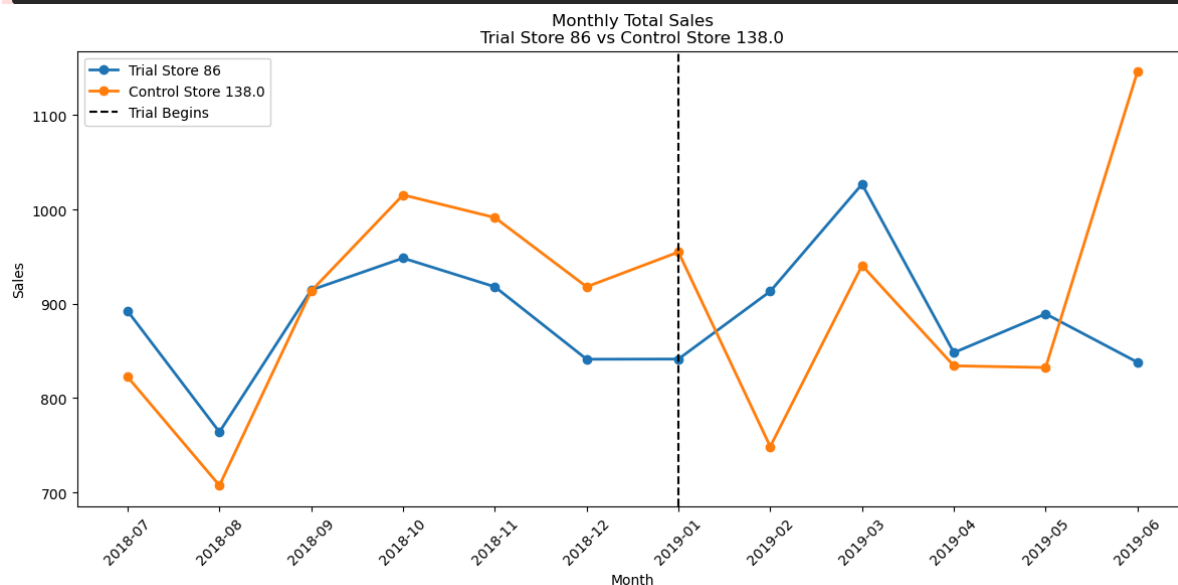
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:  
The figure layout has changed to tight  
plt.tight_layout()
```



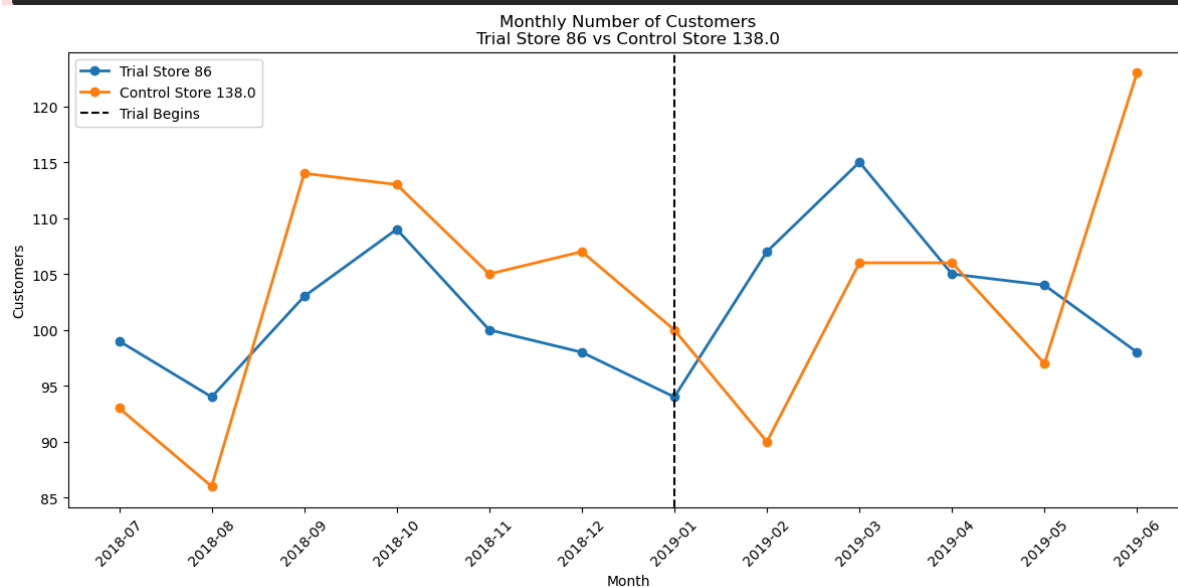
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:  
The figure layout has changed to tight  
plt.tight_layout()
```



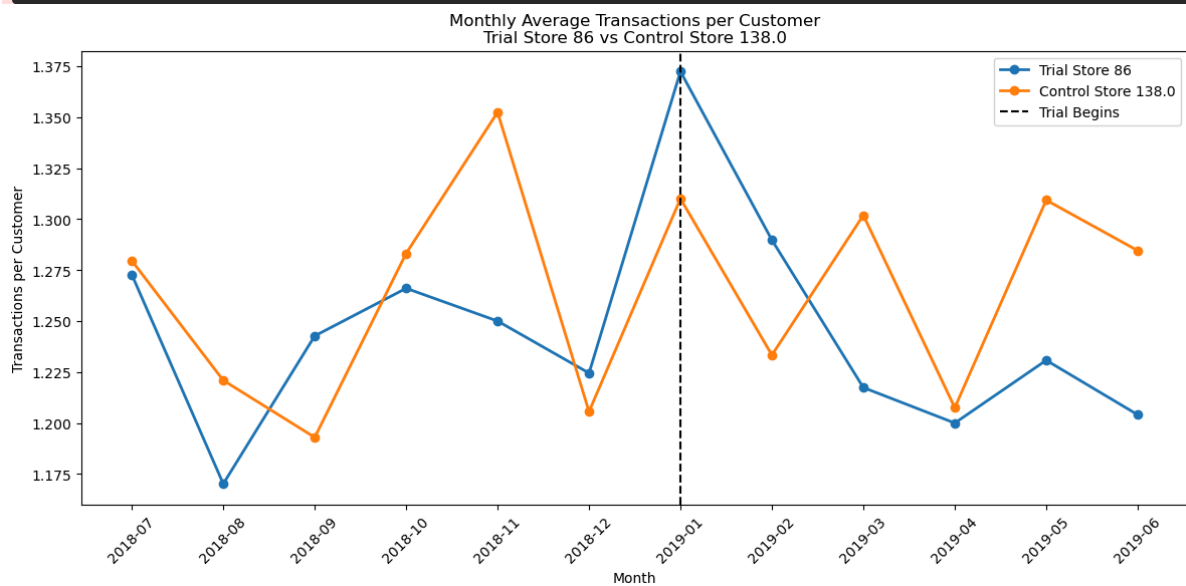
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:
The figure layout has changed to tight
plt.tight_layout()
```



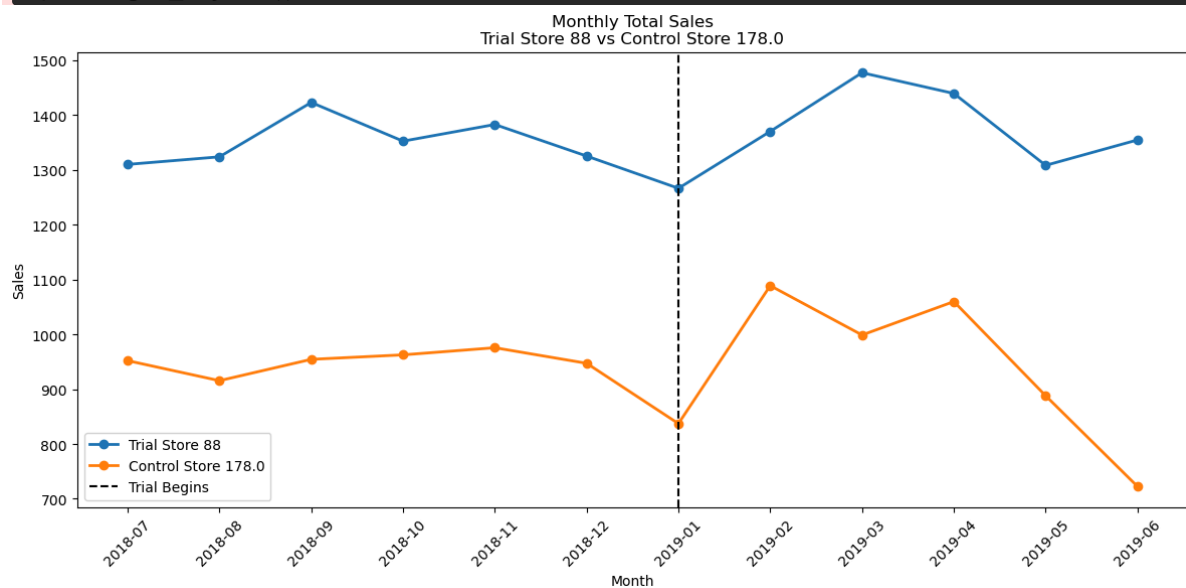
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:
The figure layout has changed to tight
plt.tight_layout()
```



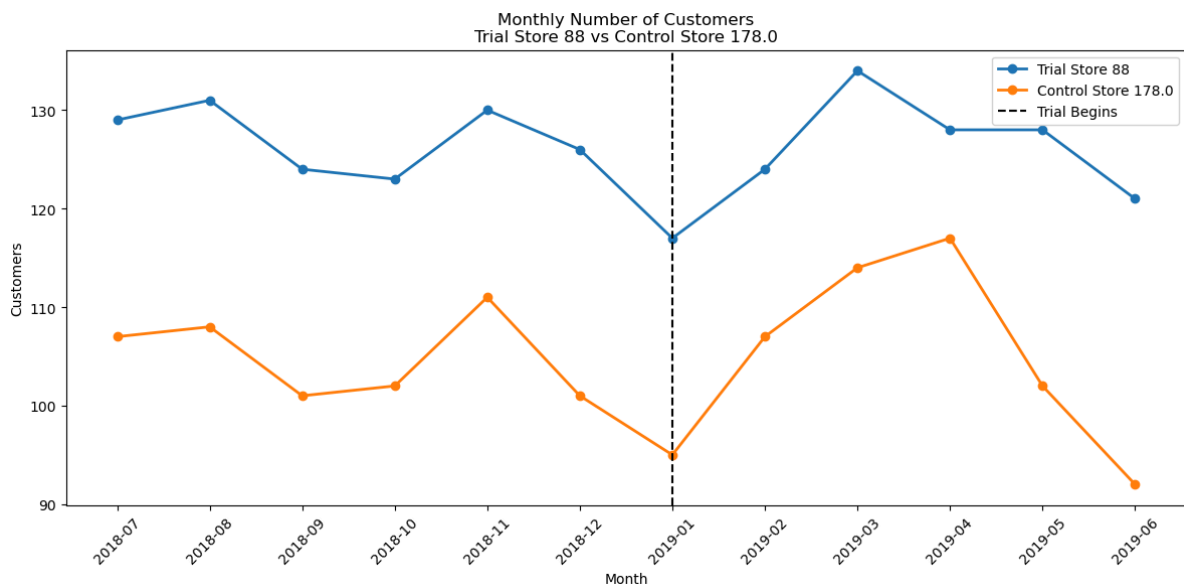
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:  
The figure layout has changed to tight  
plt.tight_layout()
```



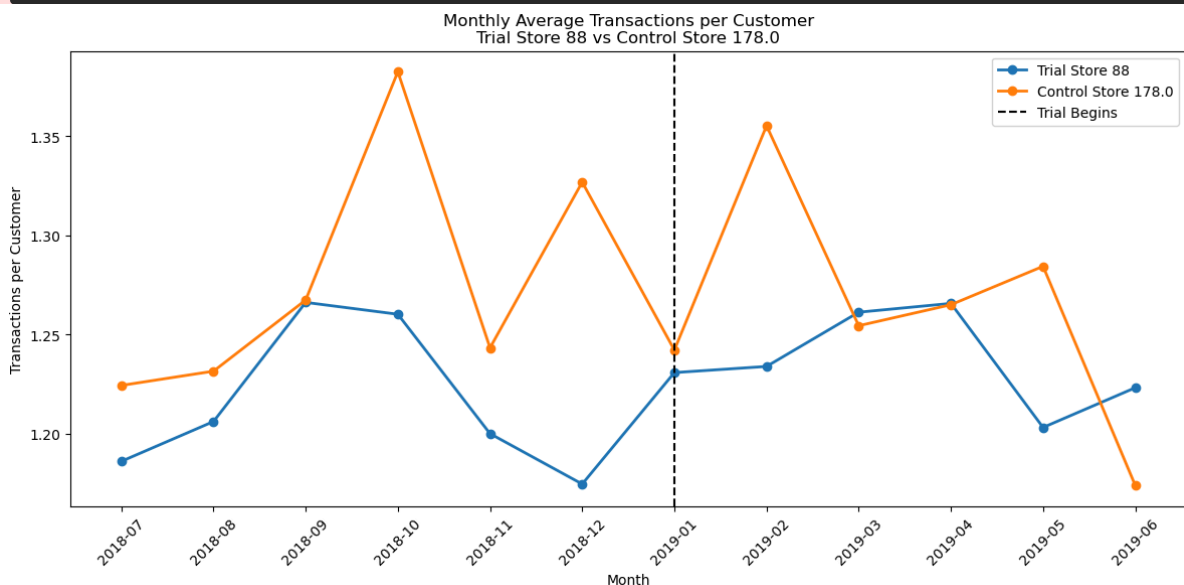
```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:  
The figure layout has changed to tight  
plt.tight_layout()
```



```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:  
The figure layout has changed to tight  
plt.tight_layout()
```



```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_13320\2360965403.py:43: UserWarning:
The figure layout has changed to tight
plt.tight_layout()
```



```
In [ ]:
```

```
In [ ]:
```

```
In [39]:
```

```
def calculate_scaling_sales(trial_store, metric):

    comparison = compare_trial_control(trial_store)

    pre_trial = comparison[
        comparison["month_id"] < "2019-02"
    ]

    scaling_factor = (
        pre_trial[f"{metric}_trial"].sum()
```

```

    /
    pre_trial[f"{metric}_control"].sum()
)

comparison[f"Scaled_{metric}_control"] = (
    comparison[f"{metric}_control"] *
    scaling_factor
)

return comparison, scaling_factor

```

In [ ]:

```

In [40]: comparison_77, scaling_factor = calculate_scaling_sales(
        77,
        "Total_sales"
    )

```

```

In [41]: comparison_77.head()

```

```

Out[41]:

```

|   | store_nbr_trial | month_id | Total_sales_trial | no_of_cust_trial | transaction_trial | Avg_txn_per_cust_trial |
|---|-----------------|----------|-------------------|------------------|-------------------|------------------------|
| 0 | 77              | 2018-07  | 296.8             | 51               | 55                | 1.0784                 |
| 1 | 77              | 2018-08  | 255.5             | 47               | 48                | 1.0212                 |
| 2 | 77              | 2018-09  | 225.2             | 42               | 44                | 1.0476                 |
| 3 | 77              | 2018-10  | 204.5             | 37               | 38                | 1.0270                 |
| 4 | 77              | 2018-11  | 245.3             | 41               | 44                | 1.0731                 |

```

In [42]: comparison_86, scaling_factor = calculate_scaling_sales(
        86,
        "Total_sales"
    )
comparison_86.head()

```

```

Out[42]:

```

|   | store_nbr_trial | month_id | Total_sales_trial | no_of_cust_trial | transaction_trial | Avg_txn_per_cust_trial |
|---|-----------------|----------|-------------------|------------------|-------------------|------------------------|
| 0 | 86              | 2018-07  | 892.20            | 99               | 126               | 1.2727                 |
| 1 | 86              | 2018-08  | 764.05            | 94               | 110               | 1.1702                 |
| 2 | 86              | 2018-09  | 914.60            | 103              | 128               | 1.2427                 |
| 3 | 86              | 2018-10  | 948.40            | 109              | 138               | 1.2660                 |
| 4 | 86              | 2018-11  | 918.00            | 100              | 125               | 1.2500                 |

```
In [43]: comparison, scaling_factor = calculate_scaling_sales(
    77,
    "Total_sales"
)

print(comparison.columns.tolist())

['store_nbr_trial', 'month_id', 'Total_sales_trial', 'no_of_cust_trial', 'transaction_trial', 'Avg_txn_per_cust_trial', 'store_nbr_control', 'Total_sales_control', 'no_of_cust_control', 'transaction_control', 'Avg_txn_per_cust_control', 'Scaled_Total_sales_control']
```

```
In [44]: comparison, scaling_factor = calculate_scaling_sales(
    77,
    "Total_sales"
)

comparison["month_label"] = comparison["month_id"].astype(str)

plt.figure(figsize=(12,6))

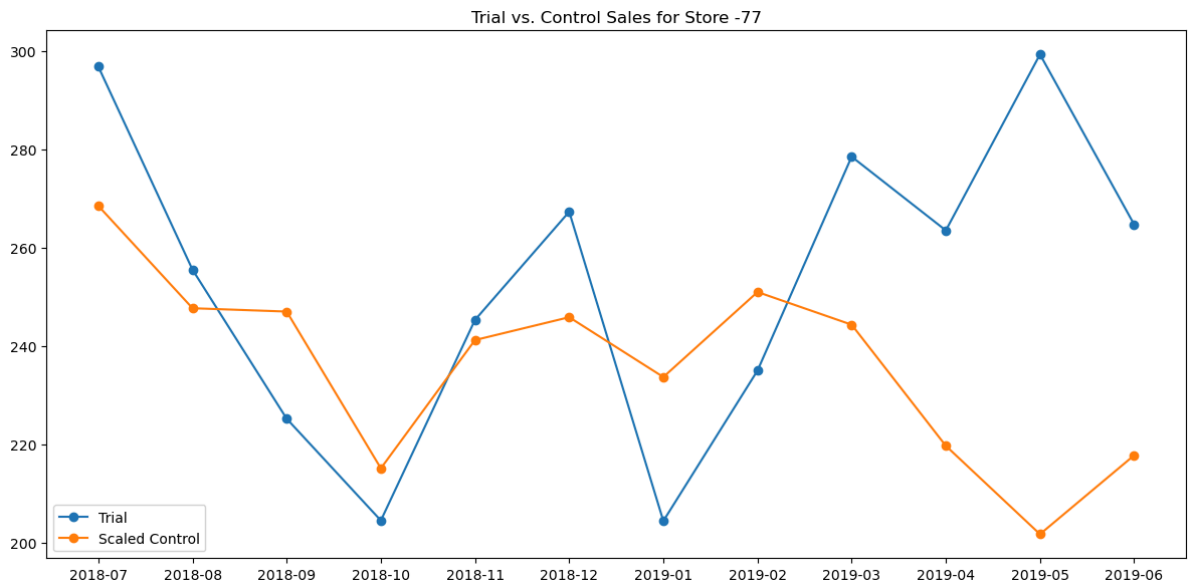
plt.plot(
    comparison["month_label"],
    comparison["Total_sales_trial"],
    marker="o",
    label="Trial"
)

plt.plot(
    comparison["month_label"],
    comparison["Scaled_Total_sales_control"],
    marker="o",
    label="Scaled Control"
)

plt.title("Trial vs. Control Sales for Store -77")
plt.legend()
plt.tight_layout()

plt.savefig(
    f"Store_77_T_C_sales.png",
    dpi=300,
    bbox_inches="tight"
)
```

```
plt.show()
```



In [ ]:

In [45]:

```
# function for statistical test

def significance_test(trial_store):

    comparison, scaling_factor = calculate_scaling_sales(
        trial_store,
        "Total_sales"
    )
    # Pre-Trial Period

    pre_trial = comparison[
        comparison["month_id"] < "2019-02"
    ].copy()

    pre_trial["pct_diff"] = (
        pre_trial["Total_sales_trial"]
        -
        pre_trial["Scaled_Total_sales_control"]
    ) / pre_trial["Scaled_Total_sales_control"]

    mean_diff = pre_trial["pct_diff"].mean()

    std_diff = pre_trial["pct_diff"].std()
```

```

upper_limit = mean_diff + 2 * std_diff

lower_limit = mean_diff - 2 * std_diff

# Trial Period

trial_period = comparison[
    comparison["month_id"] >= "2019-02"
].copy()

trial_period["pct_diff"] = (
    trial_period["Total_sales_trial"]
    -
    trial_period["Scaled_Total_sales_control"]
) / trial_period["Scaled_Total_sales_control"]

trial_period["Significant"] = (
    (trial_period["pct_diff"] > upper_limit) |
    (trial_period["pct_diff"] < lower_limit)
)

return trial_period[
    [
        "month_id",
        "Total_sales_trial",
        "Scaled_Total_sales_control",
        "pct_diff",
        "Significant"
    ]
]

```

In [46]: `significance_test(77)`

Out[46]:

|    | month_id | Total_sales_trial | Scaled_Total_sales_control | pct_diff  | Significant |
|----|----------|-------------------|----------------------------|-----------|-------------|
| 7  | 2019-02  | 235.0             | 250.971598                 | -0.063639 | False       |
| 8  | 2019-03  | 278.5             | 244.324337                 | 0.139878  | False       |
| 9  | 2019-04  | 263.5             | 219.684488                 | 0.199447  | True        |
| 10 | 2019-05  | 299.3             | 201.716891                 | 0.483763  | True        |
| 11 | 2019-06  | 264.7             | 217.760281                 | 0.215557  | True        |

In [47]: `significance_test(86)`



Out[47]:

|    | month_id | Total_sales_trial | Scaled_Total_sales_control | pct_diff  | Significant |
|----|----------|-------------------|----------------------------|-----------|-------------|
| 7  | 2019-02  | 913.2             | 724.548428                 | 0.260371  | True        |
| 8  | 2019-03  | 1026.8            | 910.379711                 | 0.127881  | False       |
| 9  | 2019-04  | 848.2             | 807.398208                 | 0.050535  | False       |
| 10 | 2019-05  | 889.3             | 805.656040                 | 0.103821  | False       |
| 11 | 2019-06  | 838.0             | 1109.180468                | -0.244487 | True        |

In [48]:

```
def visualize_significance(trial_store):

    comparison, scaling_factor = calculate_scaling_sales(
        trial_store,
        "Total_sales"
    )

    control_store = control_store_find(trial_store)

    # -----
    # Pre-Trial

    pre_trial = comparison[
        comparison["month_id"] < "2019-02"
    ].copy()

    pre_trial["pct_diff"] = (
        pre_trial["Total_sales_trial"]
        -
        pre_trial["Scaled_Total_sales_control"]
    ) / pre_trial["Scaled_Total_sales_control"]

    mean_diff = pre_trial["pct_diff"].mean()
    std_diff = pre_trial["pct_diff"].std()

    upper_limit = mean_diff + 2 * std_diff
    lower_limit = mean_diff - 2 * std_diff

    # -----
    # Confidence Interval

    comparison["Upper_Bound"] = (
        comparison["Scaled_Total_sales_control"] *
        (1 + upper_limit)
```

```
)

comparison["Lower_Bound"] = (
    comparison["Scaled_Total_sales_control"] *
    (1 + lower_limit)
)

comparison["month_label"] = comparison["month_id"].astype(str)

# Find trial start dynamically

trial_start = comparison[
    comparison["month_label"] == "2019-02"
].index[0]

# Significant months
trial_period = comparison[
    comparison["month_id"] >= "2019-02"
].copy()

significant = trial_period[
    (trial_period["Total_sales_trial"] >
     trial_period["Upper_Bound"]) |
    (trial_period["Total_sales_trial"] <
     trial_period["Lower_Bound"])
]

# -----
# Plot

plt.figure(figsize=(14,6))

plt.plot(
    comparison["month_label"],
    comparison["Total_sales_trial"],
    marker="o",
    linewidth=2,
    label=f"Trial Store {trial_store}"
)

plt.plot(
    comparison["month_label"],
```

```
comparison["Scaled_Total_sales_control"],
marker="o",
linewidth=2,
label=f"Scaled Control Store {control_store}"
)

plt.fill_between(
    comparison["month_label"],
    comparison["Lower_Bound"],
    comparison["Upper_Bound"],
    alpha=0.25,
    label="95% Confidence Interval"
)

plt.scatter(
    significant["month_label"],
    significant["Total_sales_trial"],
    color="red",
    s=120,
    zorder=5,
    label="Significant Month"
)

plt.axvline(
    x=trial_start,
    color="black",
    linestyle="--",
    linewidth=2,
    label="Trial Begins"
)

plt.title(
    f"Sales Performance Comparison\n"
    f"Trial Store {trial_store} vs Control Store {control_store}"
)

plt.xlabel("Month")
plt.ylabel("Total Sales")

plt.xticks(rotation=45)

plt.legend()
```

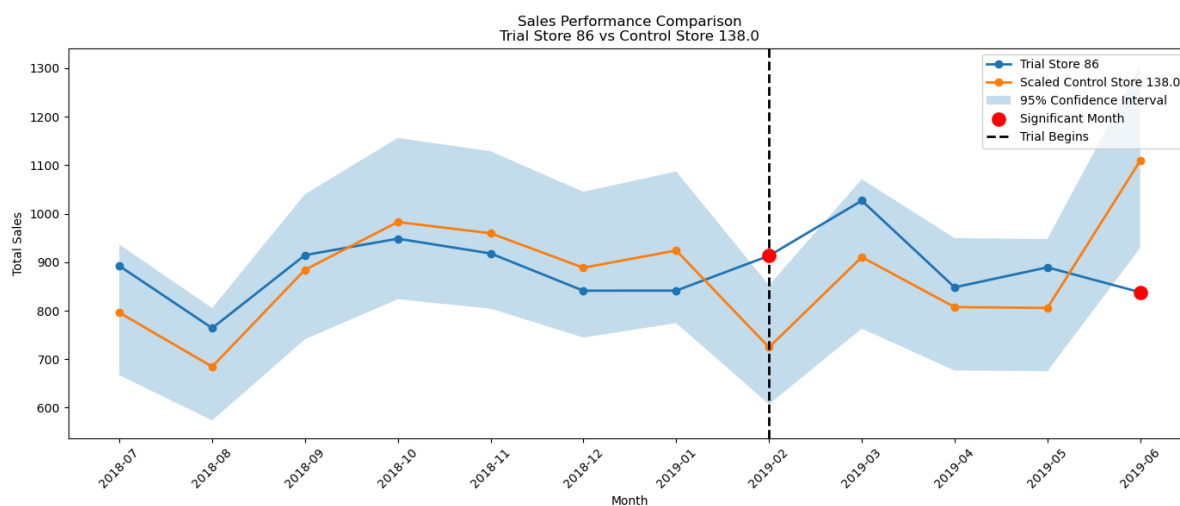
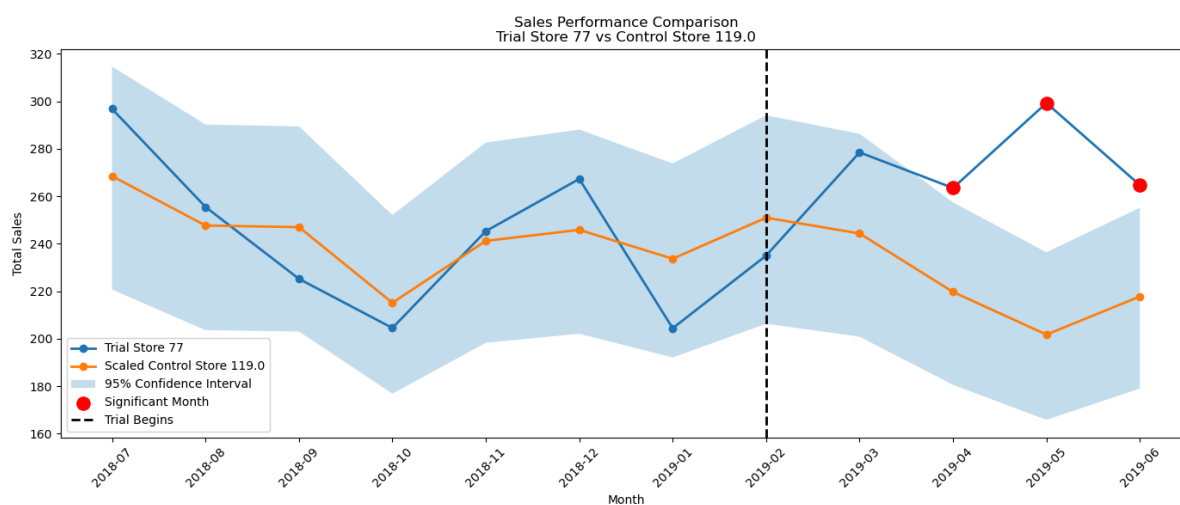
```
plt.tight_layout()

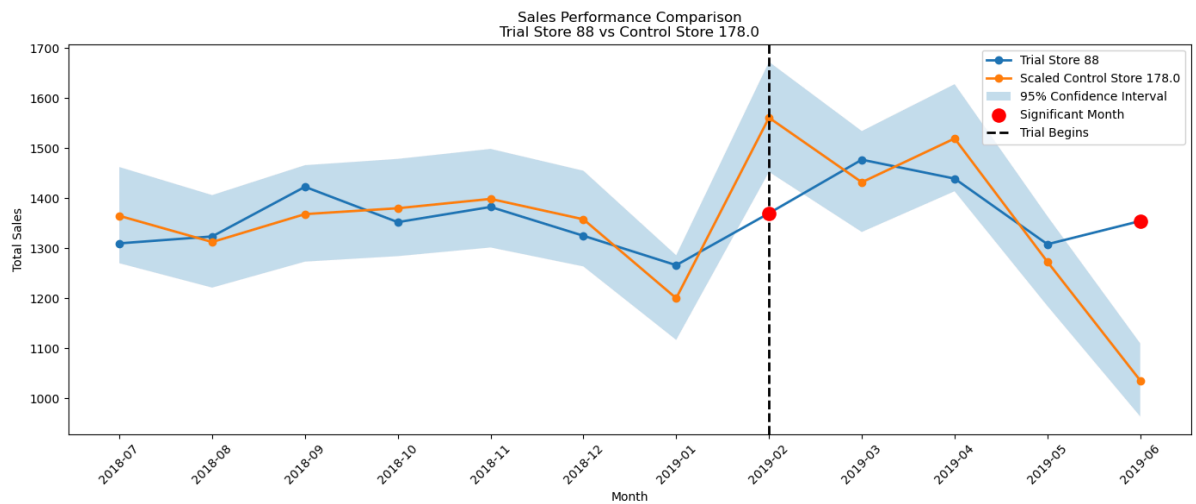
plt.savefig(
    f"Store_{trial_store}_stats.png",
    dpi=300,
    bbox_inches="tight")

plt.show()

return comparison
```

```
In [49]: for store in [77,86,88]:
        visualize_significance(store)
```





In [50]:

```
def final_summary(trial_store):

    # -----
    # Get Control Store

    control_store = control_store_find(trial_store)

    # -----
    # Get Comparison Data with Scaled Sales

    comparison, scaling_factor = calculate_scaling_sales(
        trial_store,
        "Total_sales"
    )

    # -----
    # Pre-Trial Period
    # -----

    pre_trial = comparison[
        comparison["month_id"] < "2019-02"
    ].copy()

    pre_trial["pct_diff"] = (
        pre_trial["Total_sales_trial"]
        -
        pre_trial["Scaled_Total_sales_control"]
    ) / pre_trial["Scaled_Total_sales_control"]

    mean_diff = pre_trial["pct_diff"].mean()
    std_diff = pre_trial["pct_diff"].std()
```

```
upper_limit = mean_diff + (2 * std_diff)
lower_limit = mean_diff - (2 * std_diff)

# -----
# Confidence Interval
# -----

comparison["Upper_Bound"] = (
    comparison["Scaled_Total_sales_control"]
    * (1 + upper_limit)
)

comparison["Lower_Bound"] = (
    comparison["Scaled_Total_sales_control"]
    * (1 + lower_limit)
)

# -----
# Trial Period
# -----

trial_period = comparison[
    comparison["month_id"] >= "2019-02"
].copy()

# -----
# Sales Uplift
# -----

trial_period["Sales_Uplift_%"] = (
    (
        trial_period["Total_sales_trial"]
        -
        trial_period["Scaled_Total_sales_control"]
    )
    /
    trial_period["Scaled_Total_sales_control"]
) * 100

# -----
# Significant Months
# -----

significant_months = trial_period[
    (
```

```
trial_period["Total_sales_trial"]
> trial_period["Upper_Bound"]
)
|
(
    trial_period["Total_sales_trial"]
    < trial_period["Lower_Bound"]
)
]

# -----
# Customer Uplift
# -----
customer_uplift = (
    (
        trial_period["no_of_cust_trial"].mean()
        -
        trial_period["no_of_cust_control"].mean()
    )
    /
    trial_period["no_of_cust_control"].mean()
) * 100

# -----
# Avg Transaction Uplift
# -----
txn_uplift = (
    (
        trial_period["Avg_txn_per_cust_trial"].mean()
        -
        trial_period["Avg_txn_per_cust_control"].mean()
    )
    /
    trial_period["Avg_txn_per_cust_control"].mean()
) * 100

# -----
# Average Sales Uplift
# -----
sales_uplift = trial_period["Sales_Uplift_%"].mean()

# -----
```

```

# Main Driver
# -----
if abs(customer_uplift) > abs(txn_uplift):
    driver = "Increase in Customer Numbers"
else:
    driver = "Increase in Average Transactions per Customer"

# -----
# Recommendation
# -----
if len(significant_months) >= 2:
    recommendation = (
        "The trial layout was successful. "
        "Roll out the layout to similar stores."
    )
else:
    recommendation = (
        "The trial layout did not show a statistically "
        "Rollout is not recommended."
    )

# -----
# Print Business Summary
# -----
print("=" * 70)
print(f"TRIAL STORE    : {trial_store}")
print(f"CONTROL STORE : {control_store}")
print("=" * 70)

print(f"Scaling Factor           : {scaling_factor:.3f}")
print(f"Average Sales Uplift      : {sales_uplift:.2f}%")
print(f"Average Customer Uplift   : {customer_uplift:.2f}%")
print(f"Average Transaction Uplift : {txn_uplift:.2f}%")
print(f"Significant Trial Months    : {len(significant_months)}")

if len(significant_months) > 0:
    print(
        "Months           :",
        ", ".join(significant_months["month_id"].astype(str))
    )

print(f"Primary Driver           : {driver}")

```



```
print("\nRecommendation")
print("-" * 70)
print(recommendation)

print("=" * 70)

return {
    "Trial Store": trial_store,
    "Control Store": control_store,
    "Sales Uplift (%)": round(sales_uplift, 2),
    "Customer Uplift (%)": round(customer_uplift, 2),
    "Transaction Uplift (%)": round(txn_uplift, 2),
    # "Significant Months": len(significant_months),
    "Insights": driver,
    "Recommendation": recommendation
}
```

In [51]:

```
summary = pd.DataFrame([
    final_summary(77),
    final_summary(86),
    final_summary(88)
])

summary

summary.to_csv(
    "Trial_Store_Summary.csv",
    index=False
)
```

```
=====
TRIAL STORE   : 77
CONTROL STORE : 119.0
=====

Scaling Factor      : 0.250
Average Sales Uplift : 19.50%
Average Customer Uplift : -46.76%
Average Transaction Uplift : -12.98%
Significant Trial Months : 3
Months              : 2019-04, 2019-05, 2019-06
Primary Driver      : Increase in Customer Numbers

Recommendation
-----
The trial layout was successful. Roll out the layout to similar stores.
=====

TRIAL STORE   : 86
CONTROL STORE : 138.0
=====

Scaling Factor      : 0.968
Average Sales Uplift : 5.96%
Average Customer Uplift : 1.34%
Average Transaction Uplift : -3.07%
Significant Trial Months : 2
Months              : 2019-02, 2019-06
Primary Driver      : Increase in Average Transactions per Customer

Recommendation
-----
The trial layout was successful. Roll out the layout to similar stores.
=====

TRIAL STORE   : 88
CONTROL STORE : 178.0
=====

Scaling Factor      : 1.434
Average Sales Uplift : 3.83%
Average Customer Uplift : 19.36%
Average Transaction Uplift : -2.30%
Significant Trial Months : 2
Months              : 2019-02, 2019-06
Primary Driver      : Increase in Customer Numbers

Recommendation
-----
The trial layout was successful. Roll out the layout to similar stores.
=====
```

In [ ]: